DSA LAB
VIVA QUESTION BANK

A thorough, source-aligned preparation guide

12 topics | concise answers | code walkthroughs | dry runs | edge cases | complexity and comparisons

How to use: Explain the concept, walk through the supplied code, trace an example, then state limits and complexity. Prompts marked **[General knowledge]** extend beyond explicit source details.

Built from the supplied asd.md program collection. Its first 10 topics follow the syllabus; Max Heap and Hash Table are identified in the source as content beyond syllabus. This bank is thorough but cannot guarantee every question an examiner may ask.

Contents

Linear and Binary Search 3

Bubble, Insertion, Quick Sort 5

Sparse Matrix 7

Singly Linked List 9

Polynomial Operations 11

Linked-List Stack and Postfix 13

Direct Infix Evaluation 15

Tower of Hanoi 17

Deque 19

Binary Search Tree 21

Max Heap 23

Hash Table 25

Question labels distinguish source-aligned program behavior from general extension material.

Linear and Binary Search

Program focus: Linear scan; binary search over sorted input; printed positions are one-based.

How to explain this topic aloud: When telling this aloud, explain how the active range shrinks: binary search only eliminates half because the input is sorted. Also distinguish zero-based array index from the one-based position printed by the program.

Viva questions and easy-to-say answers

Q1. How does the linear search work?

Easy-to-say answer. It compares key with arr[i] from left to right, records i+1, and stops at the first match.

Q2. Why report i+1?

Easy-to-say answer. C array indices start at zero; the program displays a human-friendly position starting at one.

Q3. What if the key occurs several times?

Easy-to-say answer. The break means this version reports the first occurrence only.

Q4. What is binary search's precondition?

Easy-to-say answer. Ascending sorted input, matching the comparisons in the program.

Q5. Explain its boundary updates.

Easy-to-say answer. If arr[mid]>key, end=mid-1; if smaller, start=mid+1; equality succeeds.

Q6. Dry run key 7 in [2,4,7,9,12].

Easy-to-say answer. mid=(0+4)/2=2; arr[2]=7; reported position is 3. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q7. Time and space?

Easy-to-say answer. Linear: O(n) worst, O(1) extra. Binary iterative: O(log n) worst, O(1) extra.

Q8. Compare them.

Easy-to-say answer. Linear handles unsorted collections; binary is faster for large sorted random-access arrays. **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Q9. What does noe<=0 guard?

Easy-to-say answer. It prevents invalid zero/negative variable-length array sizes.

Q10. How find first duplicate with binary search? [General knowledge]

Easy-to-say answer. On equality, save the match and continue in the left half until no candidates remain. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Linear and Binary Search - expanded answers

Practice question 1. Explain linear and binary search in this lab.

Suggested spoken answer. Linear search checks the array entries one by one until it finds the key or reaches the end. Binary search checks the middle entry and discards the half that cannot contain the key, so it is much faster on a large array, but only when the values are sorted.

Practice question 2. Walk through the binary search program.

Suggested spoken answer. The program starts with start=0 and end=noe-1. It calculates mid, compares arr[mid] with the key, and moves either end or start inward; if start becomes greater than end, no candidate remains. The printed position is mid+1 because the array index starts at zero.

Practice question 3. Compare their complexity and limitations.

Suggested spoken answer. Linear search takes O(n) comparisons in the worst case and does not need sorted data. Iterative binary search takes O(log n) comparisons and O(1) extra space, but its sorted-input requirement must be met before the search starts.

Bubble, Insertion, Quick Sort

Program focus: Roll numbers ascending. Bubble compares neighbors; insertion shifts larger values; quicksort uses arr[high] pivot.

How to explain this topic aloud: For an algorithm trace, say what is already in its correct region after each pass: bubble pushes a large value right, insertion grows a sorted prefix, and partition places the pivot in its final position.

Viva questions and easy-to-say answers

Q1. Describe the supplied bubble sort.

Easy-to-say answer. Each pass swaps adjacent out-of-order values; the largest remaining value travels toward the right boundary.

Q2. Bubble complexity and stability?

Easy-to-say answer. This unoptimized version makes n(n-1)/2 comparisons: O(n^2) all cases, O(1) space; stable because equals are not swapped. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q3. How does insertion sort work?

Easy-to-say answer. Save item=rollNo[i], shift larger prefix entries right, then place item at j+1.

Q4. Why save item?

Easy-to-say answer. Shifting otherwise overwrites the value being inserted.

Q5. Insertion sort bounds?

Easy-to-say answer. Best O(n) on sorted input; average/worst O(n^2); O(1) extra, stable with strict >. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q6. Explain partition.

Easy-to-say answer. Pivot is arr[high]; i tracks the next smaller-than-pivot slot; j scans; pivot is finally swapped into i.

Q7. Dry run partition [5,2,8,1].

Easy-to-say answer. Pivot 1; no scanned value is smaller; pivot swaps with 5, resulting [1,2,8,5], index 0. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q8. Quicksort bounds?

Easy-to-say answer. Average O(n log n), worst O(n^2) on poor splits; recursion stack average O(log n), worst O(n); unstable. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q9. Compare all three.

Easy-to-say answer. Insertion suits small/nearly sorted lists; bubble is simple but quadratic; quicksort is usually fast but has quadratic worst case. **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Q10. bubble best-case improvement? [General knowledge]

Easy-to-say answer. Stop after a pass with no swap; that makes the best case O(n), unlike supplied code. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Bubble, Insertion, Quick Sort - expanded answers

Practice question 1. Explain the three sorting methods used here.

Suggested spoken answer. Bubble sort repeatedly compares neighboring roll numbers and swaps an out-of-order pair. Insertion sort takes the next value and inserts it into the sorted prefix. Quicksort partitions around a pivot so smaller values go left and larger-or-equal values go right, then sorts each side recursively.

Practice question 2. Walk through the supplied quicksort partition.

Suggested spoken answer. The pivot is the last element, arr[high]. The scan uses j to inspect the range and i to mark the next slot for a value smaller than the pivot. After scanning, the pivot swaps into index i, which is its final position for that partition.

Practice question 3. Compare complexity and stability.

Suggested spoken answer. Bubble and insertion sort use constant extra storage and have O(n^2) worst-case time; insertion sort is O(n) on already sorted data. Quicksort is typically O(n log n), but this last-element-pivot version can take O(n^2) on poor splits. Bubble and insertion are stable here; quicksort is not stable.

Sparse Matrix

Program focus: SparseMatrix has dimensions, count, and MAX_TERMS=1000 Term records (row,col,value); coordinates zero-based.

How to explain this topic aloud: Explain that count means stored nonzero terms, not the total number of matrix cells. Coordinates in this program start at zero, and updating a stored value to zero removes that term.

Viva questions and easy-to-say answers

Q1. Why use a sparse representation?

Easy-to-say answer. Store only nonzero values to avoid space proportional to all matrix cells in a compact representation.

Q2. What does insertElement do?

Easy-to-say answer. Rejects invalid coordinates/full capacity; updates existing coordinate; ignores zero entries; stores new nonzero entries.

Q3. What if update sets an existing value to zero?

Easy-to-say answer. It removes that term by shifting the active suffix and reducing count.

Q4. How is the list sorted?

Easy-to-say answer. Insertion sort ascending by row then column.

Q5. What does transpose do?

Easy-to-say answer. Creates dimensions cols x rows and maps each (r,c,v) to (c,r,v), then sorts.

Q6. Dry run transpose terms (0,2,5),(1,0,-1) in 2x3.

Easy-to-say answer. Output is 3x2: sorted terms (0,1,-1),(2,0,5). **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q7. Operation costs?

Easy-to-say answer. Insert/update O(k); sort O(k^2); transpose plus sort O(k^2); display O(k), k stored terms. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q8. Edge cases?

Easy-to-say answer. Positive dimensions, valid zero-based coordinates, implicit zeros, and 1000-term capacity.

Q9. Why sort after transpose?

Easy-to-say answer. Swapping row/column can disrupt row-major order.

Q10. CSR? [General knowledge]

Easy-to-say answer. Compressed Sparse Row uses values, column indices, and row offsets; this program uses triplets. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Sparse Matrix - expanded answers

Practice question 1. Explain the sparse matrix representation.

Suggested spoken answer. Instead of storing every matrix cell as a separate active value, this program keeps a list of nonzero terms. Each term stores its row, column, and value; the matrix also stores its dimensions and the number of active terms.

Practice question 2. Explain insert/update and transpose.

Suggested spoken answer. Insertion first checks the row and column bounds. If the coordinate is already stored, its value is updated; setting it to zero removes the term, while a new zero is not stored. Transpose exchanges row and column for each term and also exchanges the matrix dimensions.

Practice question 3. What are the main limits and costs?

Suggested spoken answer. For k stored terms, finding or updating one coordinate takes O(k), and the insertion sort used to order terms takes O(k^2). Storage is capped at MAX_TERMS=1000 terms, and the program uses zero-based coordinates, so the last valid row is rows-1.

Singly Linked List

Program focus: Node contains data and next. Program inserts/deletes at beginning, middle, end; counts/displays and recursively displays reverse.

How to explain this topic aloud: Describe pointer changes in order. For deletion, first reconnect the remaining list, then free the removed node; for insertion, link the new node without losing the old next pointer.

Viva questions and easy-to-say answers

Q1. What is a singly linked list?

Easy-to-say answer. Nodes hold data and one pointer to the next node; tail->next is NULL.

Q2. Why struct Node **head?

Easy-to-say answer. Functions can change the caller's head, e.g. insertion at front or into an empty list.

Q3. How insertFirst links?

Easy-to-say answer. newNode->next=*head; then *head=newNode.

Q4. What are positions in this program?

Easy-to-say answer. One-based. Position 1 inserts at head; invalid insertion frees the unlinked allocation.

Q5. Why special case deleteEnd for one node?

Easy-to-say answer. There is no predecessor; free head and set head=NULL.

Q6. How does reverse display work?

Easy-to-say answer. Recurse to NULL, then print on unwind. It does not change links.

Q7. Complexity?

Easy-to-say answer. First insert/delete O(1); end/position operations O(n); count/display O(n); recursive reverse display uses O(n) stack. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q8. Dry run insert 8 at position 2 in 4->6.

Easy-to-say answer. Node 4 is predecessor; link 8 to 6 then 4 to 8: 4->8->6. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q9. What odd behavior for delete position <=1?

Easy-to-say answer. It deletes first node, including when position is zero/negative; a stricter interface would reject those.

Q10. why avoid recursion for huge lists? [General knowledge]

Easy-to-say answer. Recursion depth n may exhaust call stack; an iterative approach avoids that risk. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Singly Linked List - expanded answers

Practice question 1. Explain how the singly linked list works.

Suggested spoken answer. Each node contains data and a next pointer. The head points to the first node, and the last node points to NULL. To insert or delete at the head, the program changes head; that is why these functions receive the address of the head pointer.

Practice question 2. Walk through inserting and deleting a middle node.

Suggested spoken answer. For insertion, the program finds the node just before the requested one, makes the new node point to the old next node, and then links the predecessor to the new node. For deletion, it links the predecessor around the target node and frees the removed node.

Practice question 3. State the complexity and a key edge case.

Suggested spoken answer. Head insertion and deletion are O(1). Operations that must search for a position or the tail take O(n), as do counting and displaying all nodes. The code handles deleting the only node specially by freeing it and resetting head to NULL.

Polynomial Operations

Program focus: Linked list of (coefficient, exponent), descending exponents. Supports empty, insert/merge, add, display, evaluate, free.

How to explain this topic aloud: Say that the exponent identifies like terms and the coefficient is what gets combined. The descending order makes display clear, while insertTerm maintains that order as terms arrive.

Viva questions and easy-to-say answers

Q1. How is polynomial represented?

Easy-to-say answer. Each node stores integer coefficient and exponent; terms are kept in descending exponent order.

Q2. What represents zero polynomial?

Easy-to-say answer. NULL; display prints 0 and evaluation returns zero.

Q3. How are like terms handled?

Easy-to-say answer. Same exponents combine; if the sum coefficient becomes zero, the node is removed and freed.

Q4. Dry run insert 3x^2 then -3x^2.

Easy-to-say answer. Coefficients cancel; resulting list is empty. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q5. How is addition implemented?

Easy-to-say answer. Insert each term from both inputs into a fresh result; insertTerm merges matching exponents.

Q6. What is add complexity here?

Easy-to-say answer. Repeated list scans/inserts give up to O((m+n)^2); a merge of sorted lists can be O(m+n) (GK). **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q7. How is evaluation computed?

Easy-to-say answer. Sum coeff*pow(x,exp) for each node, returning double.

Q8. Why freePolynomial?

Easy-to-say answer. Each heap node must be freed to avoid memory leaks.

Q9. What input assumption exists?

Easy-to-say answer. Ordinary polynomial exponents should be nonnegative; source code does not reject negative exponents.

Q10. Compare list and array representations.

Easy-to-say answer. List stores sparse terms flexibly; arrays suit dense bounded degree polynomials and offer better locality. **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Polynomial Operations - expanded answers

Practice question 1. Explain how a polynomial is stored.

Suggested spoken answer. A node stores one coefficient and one exponent, and the nodes are kept in descending exponent order. For example, 4x^3 and -2x are stored as terms (4,3) and (-2,1). A NULL head represents the zero polynomial.

Practice question 2. How does insertion combine like terms?

Suggested spoken answer. The function searches for an existing node with the same exponent. If it finds one, it adds the new coefficient to that node; if the result is zero, it removes the node. Otherwise, it inserts the new term in descending exponent order.

Practice question 3. Explain addition, evaluation, and cost.

Suggested spoken answer. Addition copies terms from both inputs into a fresh list, so insertTerm combines matching exponents for us. Evaluation visits every term and adds coefficient times x raised to its exponent. Repeated linked-list searches make addition up to quadratic in the number of terms in this implementation.

Linked-List Stack and Postfix

Program focus: Stack is linked nodes. Postfix scanner pushes operands; for each operator pops two values and pushes result.

How to explain this topic aloud: For postfix, operands are pushed as read; when an operator arrives, pop the right operand first and the left operand second. This order is essential for subtraction and division.

Viva questions and easy-to-say answers

Q1. What is LIFO?

Easy-to-say answer. Last In, First Out: the most recently pushed value is popped first.

Q2. Push/pop complexity?

Easy-to-say answer. O(1) time each; each item uses a dynamically allocated node. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q3. What is underflow?

Easy-to-say answer. Popping an empty stack; the program checks empty top.

Q4. How evaluate postfix?

Easy-to-say answer. Push operands; on operator pop b then a, compute a op b, push result.

Q5. Why pop b before a?

Easy-to-say answer. For subtraction/division order matters: 8 2 - means 8-2, not 2-8.

Q6. Dry run 5 2 3 * +.

Easy-to-say answer. 2*3=6, then 5+6=11. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q7. What should remain at end?

Easy-to-say answer. Exactly one value. Other stack sizes indicate malformed expression.

Q8. Complexity?

Easy-to-say answer. O(t) time and O(t) worst-case stack for t tokens. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q9. What operand limitation is in supplied code?

Easy-to-say answer. Character scanning supports single-digit operands; multi-digit/signed values need token parsing.

Q10. What division caveat?

Easy-to-say answer. Integer division truncates (7/2 is 3); reject division by zero.

Q11. why postfix is convenient? [General knowledge]

Easy-to-say answer. It encodes precedence/grouping without parentheses and supports direct stack evaluation. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Linked-List Stack and Postfix - expanded answers

Practice question 1. Explain the linked-list stack.

Suggested spoken answer. A stack follows last in, first out order. The top pointer identifies the most recently pushed node; push adds a node at the top, and pop removes and returns that node, so both operations take constant time.

Practice question 2. Explain postfix evaluation with an example.

Suggested spoken answer. Read the expression from left to right and push each operand. When an operator appears, pop the right operand first and then the left operand, calculate, and push the result. For 5 2 3 * +, first 2*3 gives 6, then 5+6 gives 11.

Practice question 3. What errors and representation limits should you mention?

Suggested spoken answer. A valid expression should leave exactly one result on the stack, and division by zero must be rejected. The supplied character-based scanner is suitable for single-digit operands; multi-digit numbers need token-based parsing. Integer division also truncates the fractional part.

Direct Infix Evaluation

Program focus: Two global arrays hold values/operators (MAX=100); skips whitespace, parses multi-digit positive integers, handles parentheses and + - * /.

How to explain this topic aloud: Explain that values and operators wait on separate stacks. Precedence decides when to calculate, and parentheses force the enclosed operations to finish before the outer expression continues.

Viva questions and easy-to-say answers

Q1. How evaluate directly?

Easy-to-say answer. Value and operator stacks; apply pending operators by precedence and at closing parentheses.

Q2. What precedence?

Easy-to-say answer. + and - level 1; * and / level 2.

Q3. Why reduce while precedence >= current?

Easy-to-say answer. Equal-precedence operators are left-associative, so earlier operation must be applied first.

Q4. Dry run 2+3*4.

Easy-to-say answer. Multiplication first gives 12; then addition gives 14. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q5. Dry run 8-3-2.

Easy-to-say answer. Left-associative: (8-3)-2=3. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q6. How parse multi-digit values?

Easy-to-say answer. Accumulate num=num*10+digit while characters are digits.

Q7. What happens on division by zero?

Easy-to-say answer. Prints an error and exits.

Q8. Does it handle unary minus?

Easy-to-say answer. No explicit support; leading negative numbers and 3*(-2) are unsafe.

Q9. Are malformed expressions fully validated?

Easy-to-say answer. No. Mismatched parentheses/operand shortages are not robustly rejected; fixed stacks also lack overflow checks.

Q10. Complexity?

Easy-to-say answer. O(n) time and O(n) stack for n input characters. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q11. exponentiation associativity? [General knowledge]

Easy-to-say answer. Usually right-associative, requiring a different reduction rule; not supported here. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Direct Infix Evaluation - expanded answers

Practice question 1. Explain the two-stack infix evaluator.

Suggested spoken answer. One stack holds values and another holds operators. As the program reads the expression, it applies operators with higher or equal precedence before pushing a new operator; a closing parenthesis causes operations to be applied until the matching opening parenthesis is reached.

Practice question 2. Walk through 2 + 3 * 4.

Suggested spoken answer. The program pushes 2, then the plus operator, then 3. When it reads multiplication, plus has lower precedence so multiplication is held for later; after reading 4, it evaluates 3*4=12 and then 2+12=14.

Practice question 3. What syntax does this code support, and what does it miss?

Suggested spoken answer. It skips whitespace, reads multi-digit nonnegative integers, handles parentheses, and supports +, -, *, and /. It checks division by zero, but does not safely implement unary minus or full malformed-expression checks, and its fixed-size stacks need bounds checks for robust use.

Tower of Hanoi

Program focus: Recursive towers_of_hanoi(n,a,b,c); global cnt numbers moves; main rejects n<=0.

How to explain this topic aloud: Explain the recursive plan in three moves: move the smaller stack aside, move the largest disk, then move the smaller stack onto it. Each recursive call changes the roles of the towers.

Viva questions and easy-to-say answers

Q1. State the rules.

Easy-to-say answer. Move one disk at a time, only top disk, and never place larger on smaller.

Q2. What is base case?

Easy-to-say answer. For one disk, move source directly to destination and return.

Q3. What is recursive case?

Easy-to-say answer. Move n-1 to auxiliary; move largest to destination; move n-1 from auxiliary to destination.

Q4. Dry run n=2.

Easy-to-say answer. Disk 1 source->auxiliary; disk 2 source->destination; disk 1 auxiliary->destination. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q5. Moves recurrence and closed form?

Easy-to-say answer. T(n)=2T(n-1)+1; T(n)=2^n-1.

Q6. Complexities?

Easy-to-say answer. O(2^n) moves/time and O(n) recursion stack. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q7. Purpose of cnt?

Easy-to-say answer. Global counter shared by recursive calls to number moves.

Q8. Why roles of towers change?

Easy-to-say answer. Each subproblem has different source, auxiliary, destination roles.

Q9. What limits practical input?

Easy-to-say answer. Output grows exponentially; very large depth can exhaust stack.

Q10. 4-peg variant? [General knowledge]

Easy-to-say answer. Requires a different strategy such as Frame-Stewart; not the supplied 3-peg algorithm. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Tower of Hanoi - expanded answers

Practice question 1. Explain the recursive Tower of Hanoi plan.

Suggested spoken answer. To move n disks, first move the top n-1 disks from the source to the auxiliary tower. Then move the largest disk to the destination, and finally move those n-1 disks from the auxiliary tower onto it.

Practice question 2. Trace the two-disk case.

Suggested spoken answer. Move disk 1 from Tower 1 to Tower 2, move disk 2 from Tower 1 to Tower 3, then move disk 1 from Tower 2 to Tower 3. That uses three moves and follows the rule that a larger disk is never placed on a smaller one.

Practice question 3. Give the recurrence and complexity.

Suggested spoken answer. The recurrence is T(n)=2T(n-1)+1 because two smaller problems are solved and one largest-disk move is made. This gives 2^n-1 moves, so running time is exponential, while recursive call depth is O(n).

Deque

Program focus: Circular array MAX=5; front/rear indices; insert/delete from either end; overflow/underflow messages.

How to explain this topic aloud: The deque's logical order may wrap around the end of the array. Explain front and rear as circular indices, not as positions that must always increase from left to right.

Viva questions and easy-to-say answers

Q1. Define deque.

Easy-to-say answer. Double-ended queue supporting insertion and deletion at front and rear.

Q2. What means front=rear=-1?

Easy-to-say answer. Empty deque.

Q3. How detect full?

Easy-to-say answer. front==0 && rear==MAX-1, or front==rear+1.

Q4. How insertFront wraps?

Easy-to-say answer. If front is zero, set MAX-1; otherwise decrement.

Q5. How insertRear wraps?

Easy-to-say answer. If rear is MAX-1, set zero; otherwise increment.

Q6. How handle deleting last item?

Easy-to-say answer. Reset both indices to -1.

Q7. Dry run MAX=5: rear insert 10,20; deleteFront; front insert 5.

Easy-to-say answer. After deletion [20]; front insert wraps to index 4, logical deque [5,20]. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q8. Complexities?

Easy-to-say answer. End insert/delete O(1); display O(n); fixed storage O(MAX). **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q9. What is sentinel caveat?

Easy-to-say answer. Returning -1 for failure conflicts if -1 can be legitimate data; use separate status/output.

Q10. Compare deque and queue.

Easy-to-say answer. Queue usually inserts rear/deletes front; deque supports both ends for each operation. **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Q11. restricted deque variants? [General knowledge]

Easy-to-say answer. Input-restricted permits insertion at one end only; output-restricted permits deletion at one end only. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Deque - expanded answers

Practice question 1. Explain a deque and this implementation.

Suggested spoken answer. A deque, or double-ended queue, allows insertion and deletion at both ends. The program stores it in a circular array of size five; front and rear move forward or wrap back to index zero or four.

Practice question 2. How do empty, full, and wraparound work?

Suggested spoken answer. Both indices are -1 when empty. It is full when the indices are adjacent in the circular layout or occupy the two physical ends in order. When an index moves past an array boundary, the code wraps it around rather than shifting stored items.

Practice question 3. State the operation costs and one design limitation.

Suggested spoken answer. Insertion and deletion at either end take O(1); displaying all elements takes O(n). The array has a fixed capacity of five, and -1 is used as a deletion error sentinel, so it could be confused with valid data unless the interface separates status from the removed value.

Binary Search Tree

Program focus: Node has data,L,R. Greater goes right; duplicates go left. Has longest-path count, minimum, mirror, search, traversals.

How to explain this topic aloud: For a normal BST, each comparison chooses one branch. Mention that this code sends duplicates left and that mirroring swaps the order, so normal BST search should not be used on the mirrored tree.

Viva questions and easy-to-say answers

Q1. BST invariant?

Easy-to-say answer. Left-side keys are <= node here; right-side keys are > node, because duplicate keys go left.

Q2. What does inorder produce?

Easy-to-say answer. Nondecreasing keys before mirroring.

Q3. Name traversal orders.

Easy-to-say answer. Inorder L-root-R; preorder root-L-R; postorder L-R-root.

Q4. How search?

Easy-to-say answer. Compare key and move left/right until found or NULL.

Q5. How find minimum?

Easy-to-say answer. Follow left pointers; empty tree is reported.

Q6. What does height return?

Easy-to-say answer. Number of nodes on longest path: NULL=0 and a leaf=1.

Q7. Dry run insert 8,3,10,1.

Easy-to-say answer. Inorder 1,3,8,10; preorder 8,3,1,10. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q8. What does mirror do?

Easy-to-say answer. Recursively swaps left/right pointers at every node.

Q9. Can search still use BST ordering after mirror?

Easy-to-say answer. No; mirrored ordering is reversed, so ordinary search assumptions fail.

Q10. Complexity?

Easy-to-say answer. Search/insert/min O(h): balanced average O(log n), skewed worst O(n); traversal/height O(n). **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q11. how maintain balance? [General knowledge]

Easy-to-say answer. AVL or red-black trees use rotations to keep height logarithmic. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Binary Search Tree - expanded answers

Practice question 1. Explain the BST rule and insertion in this code.

Suggested spoken answer. At each node, smaller or equal values go left and greater values go right; this implementation therefore places duplicates on the left. Insertion compares the new value at each node and follows one child link until it reaches an empty position.

Practice question 2. Explain traversals and minimum.

Suggested spoken answer. Inorder visits left subtree, node, right subtree, so it displays values in nondecreasing order before mirroring. Preorder visits the node first, and postorder visits it last. The minimum is found by following left links until there is no left child.

Practice question 3. Explain height and worst-case performance.

Suggested spoken answer. This program defines height as the number of nodes on the longest root-to-leaf path: an empty tree has height zero and a leaf has height one. Search and insertion take O(h); if the tree becomes a chain, h is n and those operations take O(n).

Max Heap

Program focus: Array heapify; children 2i+1/2i+2. Build runs heapify from n/2-1 down to zero.

How to explain this topic aloud: Explain the parent-child rule rather than calling the heap sorted. During heapify, the largest of a node and its children moves upward; bottom-up construction starts at the last internal node.

Viva questions and easy-to-say answers

Q1. Define max heap.

Easy-to-say answer. Complete binary tree where every parent is >= its children.

Q2. Is heap sorted?

Easy-to-say answer. No, only parent-child order is guaranteed.

Q3. Why array representation?

Easy-to-say answer. Complete-tree layout gives compact level-order storage and formula-based indices.

Q4. What does heapify do?

Easy-to-say answer. Select largest among parent/children, swap when needed, recurse into affected child.

Q5. Why start at n/2-1?

Easy-to-say answer. Higher indices are leaves; last internal node is n/2-1.

Q6. Dry run [4,10,3,5,1].

Easy-to-say answer. Heapify index 1 unchanged; root swaps 4/10, then 4/5: [10,5,3,4,1]. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q7. Heapify/build bounds?

Easy-to-say answer. Heapify O(log n); bottom-up buildMaxHeap O(n). **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q8. Space?

Easy-to-say answer. O(n) array, plus O(log n) recursive heapify stack.

Q9. Common index bug?

Easy-to-say answer. Using one-based formulas with zero-based array; here children are 2i+1 and 2i+2.

Q10. Compare heap and BST.

Easy-to-say answer. Heap root provides max quickly, arbitrary search O(n); BST supports ordered search O(h). **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Q11. heap sort? [General knowledge]

Easy-to-say answer. Repeated root extraction and heapify gives O(n log n), in-place, generally unstable. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Max Heap - expanded answers

Practice question 1. Explain the max-heap property and array layout.

Suggested spoken answer. A max heap is a complete binary tree where every parent is at least as large as its children. In the zero-based array used here, node i has children at 2i+1 and 2i+2; the largest value is at index zero.

Practice question 2. Explain heapify and buildMaxHeap.

Suggested spoken answer. Heapify compares a node with its children and swaps it with the largest child if the heap rule is broken, then continues down that subtree. BuildMaxHeap starts at the last internal node, n/2-1, and works backward to the root.

Practice question 3. State complexity and distinguish heap from sorted array.

Suggested spoken answer. Heapify takes O(log n), while bottom-up buildMaxHeap takes O(n). A heap is not fully sorted: it only guarantees that each parent is no smaller than its children, which is enough to expose the maximum at the root.

Hash Table

Program focus: SIZE=10; key%SIZE; linear probing with wraparound. EMPTY=-1, DELETED=-2.

How to explain this topic aloud: Describe a collision as a shared starting slot. Linear probing checks the next slot in a circle; a deleted marker must remain distinct from a never-used empty slot so searches do not stop too early.

Viva questions and easy-to-say answers

Q1. What is hashing?

Easy-to-say answer. Map keys to indices to support fast expected lookup.

Q2. What is collision?

Easy-to-say answer. Distinct keys map to the same index.

Q3. How does linear probing work?

Easy-to-say answer. Check subsequent slots modulo table size until key, available slot, or full cycle.

Q4. Dry run insert 12,22,32.

Easy-to-say answer. All hash to 2; occupy slots 2,3,4. **To expand your answer:** In an oral trace, name each intermediate index or stack/list state before giving the final result.

Q5. Why stop search at EMPTY but not DELETED?

Easy-to-say answer. EMPTY proves no later displaced key exists along probe; DELETED tombstone preserves the chain.

Q6. How handle full table?

Easy-to-say answer. After a complete cycle to starting index, report full.

Q7. Complexity?

Easy-to-say answer. Expected O(1) at moderate load; worst O(SIZE) for search/insert/delete. **To expand your answer:** State what n (or k, h, or t) measures, then give the worst case as well as the usual case when they differ.

Q8. Why reserve -1 and -2?

Easy-to-say answer. They encode empty/deleted states and cannot also be valid keys.

Q9. What if keys can be negative?

Easy-to-say answer. C remainder may be negative and index invalid; normalize modulo. Current code assumes nonnegative keys.

Q10. Why not replace deletion with EMPTY?

Easy-to-say answer. It could make colliding keys later in the probe sequence unreachable.

Q11. Compare probing and chaining.

Easy-to-say answer. Probing stores entries in array with good locality but clustering; chaining uses per-bucket lists and extra allocations. **To expand your answer:** Finish by connecting the choice to the situation: input order, expected size, memory, or operation pattern.

Q12. define load factor and primary clustering. [General knowledge]

Easy-to-say answer. Load factor n/table-size; high load increases probes. Primary clustering is growth of contiguous occupied runs in linear probing. **To expand your answer:** This is an extension beyond the supplied implementation; first describe what the program does, then give this broader idea.

Hash Table - expanded answers

Practice question 1. Explain hashing and collision handling in this code.

Suggested spoken answer. The hash function maps a key to key modulo ten. If two keys map to the same slot, linear probing checks the next slot, wrapping around the array until it finds the key or a slot it can use.

Practice question 2. Why does deletion use a DELETED marker?

Suggested spoken answer. Suppose keys collide and one is placed after another. If the earlier key is deleted and its slot becomes EMPTY, a later search may stop there and miss the key farther along. DELETED marks the slot as reusable while letting search continue.

Practice question 3. State performance and key limitations.

Suggested spoken answer. With a reasonable spread of keys and free space, operations are usually close to O(1); in the worst case they scan all ten slots. The table reserves -1 and -2 for markers, and the modulo expression assumes nonnegative keys.

Rapid Revision

Topic	Quick recall
Search	Linear O(n); binary O(log n), sorted input.
Sort	Bubble O(n^2) here; insertion best O(n); quicksort average O(n log n), worst O(n^2).
Sparse matrix	Triplet terms; zero-based row/column; capacity 1000; sort O(k^2).
Linked list	Pointer-to-pointer can change head; reverse display recursion uses O(n) stack.
Polynomial	Sorted exponents; equal terms combine; zero coefficient removed.
Stack/postfix	LIFO; pop right then left; source scanner single-digit.
Infix	Two stacks; +,- before *,/; unary minus not supported.
Hanoi	2^n-1 moves; O(n) recursion space.
Deque	Circular array MAX=5; end ops O(1).
BST	Inorder sorted until mirrored; worst operations O(n).
Max heap	Children 2i+1/2i+2; build O(n).
Hash	Modulo 10, linear probing, DELETED tombstone.

Mock Viva

Answer aloud; checkpoints are concise reminders rather than scripts.

1. Why does binary search need sorted data?

Checkpoint. Half-elimination decisions depend on ordering.

2. Trace postfix 6 2 3 + *.

Checkpoint. 2+3=5, then 6*5=30.

3. Why is DELETED different from EMPTY?

Checkpoint. Tombstone keeps later colliding keys searchable.

4. Why pass a pointer to list head?

Checkpoint. An insertion/deletion may change caller's head.

5. Hanoi moves for four disks?

Checkpoint. 15.

6. Why can a BST be slow?

Checkpoint. Skewed height n gives O(n) operations.

7. What does heapify restore?

Checkpoint. Max-parent relation for that subtree.

8. Deque full condition?

Checkpoint. front==0 && rear==MAX-1 or front==rear+1.

9. Name an infix evaluator limitation.

Checkpoint. Unary minus unsupported; limited unchecked stacks and weak malformed-input checks.

10. Compare insertion and quicksort for nearly sorted data.

Checkpoint. Insertion can run O(n); quicksort may hit O(n^2) from poor splits.

11. What is a sparse term?

Checkpoint. A stored nonzero (row,column,value).

12. How many values should valid postfix leave?

Checkpoint. Exactly one.

Oral answer checklist

Define the idea; explain the supplied implementation; dry-run a small case; handle boundaries; give time and space; compare a related structure; state implementation limitations honestly.

Source note: Questions are grounded in the supplied asd.md programs. [General knowledge] prompts are explicitly marked.